

实验二外接 RAM 接口设计第一部分

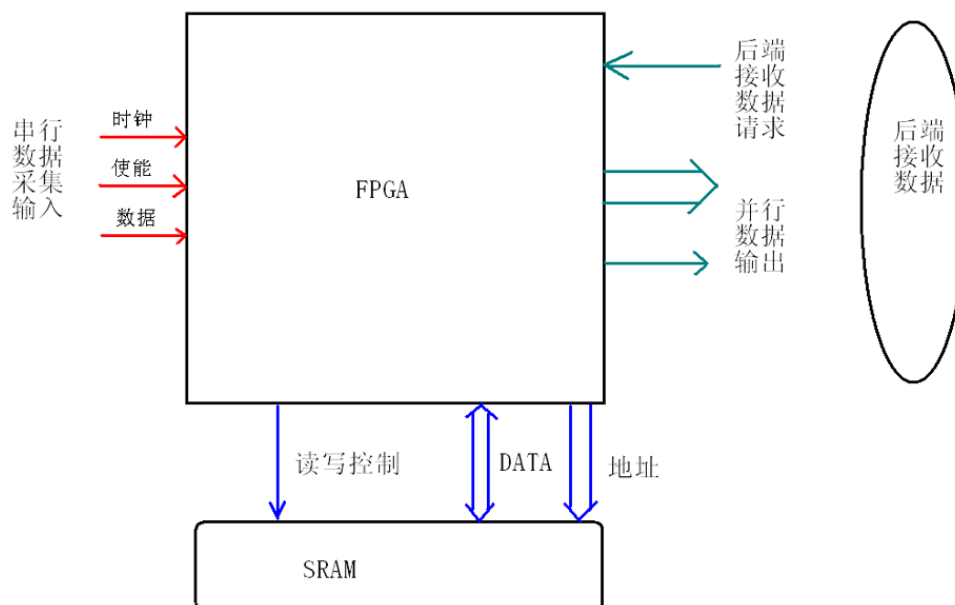
一、实验目的

1. 学习数字系统模块分割
2. 掌握外接存储器接口技术, 设计一个对片外 SRAM 进行读写的控制接口。

二、实验步骤

1. 对于一个串行输入, SRAM 读写和并行输出的系统框图, 补充系统可能需要的设计部分。

框图:



2. 学习分析 memtest.vhd 代码的结构。

```

1  library IEEE;
2  use IEEE.std_logic_1164.all;
3  use ieee.std_logic_arith.all;
4  use ieee.std_logic_unsigned.all;
5
6  -- 内存测试控制器实体
7  entity memtest is
8  port (
9      rst      : in std_logic;      -- 复位信号，低电平有效
10     clk       : in std_logic;      -- 系统时钟
11     den       : in std_logic;      -- 串行输入使能，高电平有效时接收数据
12     rxd       : in std_logic;      -- 串行输入数据，逐位输入
13     outen     : in std_logic;      -- 输出数据请求，高电平请求读取内存数据
14     rdmem     : out std_logic;      -- 读内存控制信号，低电平有效
15     wrmem     : out std_logic;      -- 与内存控制信号，低电平有效
16     csmem     : out std_logic;      -- 内存片选信号，高电平有效
17     memdata   : inout std_logic_vector(7 downto 0); -- 内存数据双向总线
18     memaddr   : out std_logic_vector(2 downto 0); -- 内存地址线（3位，可寻址8个位置）
19     dataout   : out std_logic_vector(7 downto 0); -- 并行数据输出
20     dataclkout : out std_logic;      -- 数据输出同步时钟
21 );
22 end memtest;
23
24 architecture behav of memtest is
25
26     -- 状态机状态定义（独热码onehot编码）
27     constant s0 : std_logic_vector(2 downto 0) := "001"; -- 状态0：空闲状态
28     constant s1 : std_logic_vector(2 downto 0) := "010"; -- 状态1：与内存状态
29     constant s2 : std_logic_vector(2 downto 0) := "100"; -- 状态2：读内存状态
30
31     signal ss : std_logic_vector(2 downto 0); -- 当前状态寄存器
32     signal rdmemaddr, wrmemaddr : std_logic_vector(2 downto 0); -- 读/与地址计数器
33     signal rxdcnt : std_logic_vector(3 downto 0); -- 串行数据计数器（0-8）
34     signal rdmemdata, wrmemdata : std_logic_vector(7 downto 0); -- 读/与数据寄存器
35     signal wrmem_s, wrdy, dataclkout_s : std_logic; -- 内部控制信号
36
37 begin
38
39     -- 进程1：带使能的串行数据移位寄存器
40     -- 功能：将串行输入数据rxd移位转换为8位并行数据
41     process(rst, clk)
42     begin
43         if rst = '0' then
44             wrmemdata <= (others => '0');
45         elsif clk'event and clk = '1' then
46             if den = '1' then -- 串行输入使能有效时进行移位，den一次为高八个时钟周期
47                 -- 逐位右移，新数据从最低位移入
48                 wrmemdata(7) <= wrmemdata(6);
49                 wrmemdata(6) <= wrmemdata(5);
50                 wrmemdata(5) <= wrmemdata(4);
51                 wrmemdata(4) <= wrmemdata(3);
52                 wrmemdata(3) <= wrmemdata(2);
53                 wrmemdata(2) <= wrmemdata(1);
54                 wrmemdata(1) <= wrmemdata(0);
55                 wrmemdata(0) <= rxd; -- 新数据位移入最低位
56             end if;
57         end if;
58     end process;
59
60     -- 进程2：串行数据计数器
61     -- 功能：计数已接收的串行数据位数，用于判断何时完成一个字节的接收
62     process(rst, clk)
63     begin
64         if rst = '0' then
65             rxdcnt <= (others => '0');
66         elsif clk'event and clk = '1' then
67             if den = '1' then -- 串行输入使能有效时计数
68                 if rxdcnt = 8 then -- 已接收8位数据，计数器保持
69                     rxdcnt <= rxdcnt;
70                 else -- 否则计数器加1
71                     rxdcnt <= rxdcnt + 1;
72                 end if;
73             else -- 串行输入使能无效时计数器清零
74                 rxdcnt <= (others => '0');
75             end if;
76         end if;
77     end process;
78

```

```

79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158

-- 进程3: 状态机控制逻辑
-- 功能: 根据输入信号控制状态转移
process(rst,clk)
begin
    if rst = '0' then
        ss <= s0; -- 复位时进入空闲状态
    elsif clk'event and clk = '1' then
        case ss is
            when s0 => -- 空闲状态
                if wrdy = '1' then -- 与数据准备好, 转移到与状态
                    ss <= s1;
                elsif outen = '1' then -- 输出请求有效, 转移到读状态
                    ss <= s2;
                end if;
            when s1 => -- 与内存状态, 完成后返回空闲状态 (s1,s2只存在一个时钟节拍)
                ss <= s0;
            when s2 => -- 读内存状态, 完成后返回空闲状态
                ss <= s0;
            when others => -- 异常状态处理, 返回空闲状态
                ss <= s0;
        end case;
    end if;
end process;

-- 进程4: 与准备好信号生成
-- 功能: 生成wrdy信号, 指示已接收完一个完整字节
process(rst,clk)
begin
    if rst = '0' then
        wrdy <= '0'; -- 复位时与准备好信号无效
    elsif clk'event and clk = '1' then
        if ss = s1 then -- 处于与状态时清除准备好信号
            wrdy <= '0';
        else
            if rxdcnt = 8 then -- 已接收8位数据, 设置准备好信号
                wrdy <= '1';
            else -- 否则清除准备好信号
                wrdy <= '0';
            end if;
        end if;
    end if;
end process;

-- 组合逻辑: 内存控制信号生成
wrmen_s <= '0' when ss = s1 else '1'; -- 与内存信号 (当条件ss = s1为真时, wrmen_s被赋值为'0'; 否则赋值为'1')
rdmem <= '0' when ss = s2 else '1'; -- 读内存信号 (当条件ss = s2为真时, rdmem_s被赋值为'0'; 否则赋值为'1')
csmem <= '1' when ss = s1 or ss = s2 else '0'; -- (当条件ss = s1或ss = s2为真时, 被赋值为'1', 否则则为'0')

-- 进程5: 数据输出同步时钟生成, 并行输出时为了后端对信号有所区分而需要生成同步的时钟信号
process(rst,clk)
begin
    if rst = '0' then
        dataclkout_s <= '0'; -- 复位时同步时钟无效
    elsif clk'event and clk = '1' then
        if ss = s2 then -- 读状态时产生同步时钟脉冲
            dataclkout_s <= '1';
        else
            dataclkout_s <= '0';
        end if;
    end if;
end process;

-- 进程6: 同步时钟输出寄存器
process(clk)
begin
    if clk'event and clk = '1' then
        dataclkout <= dataclkout_s; -- 寄存器输出同步时钟
    end if;
end process;

```

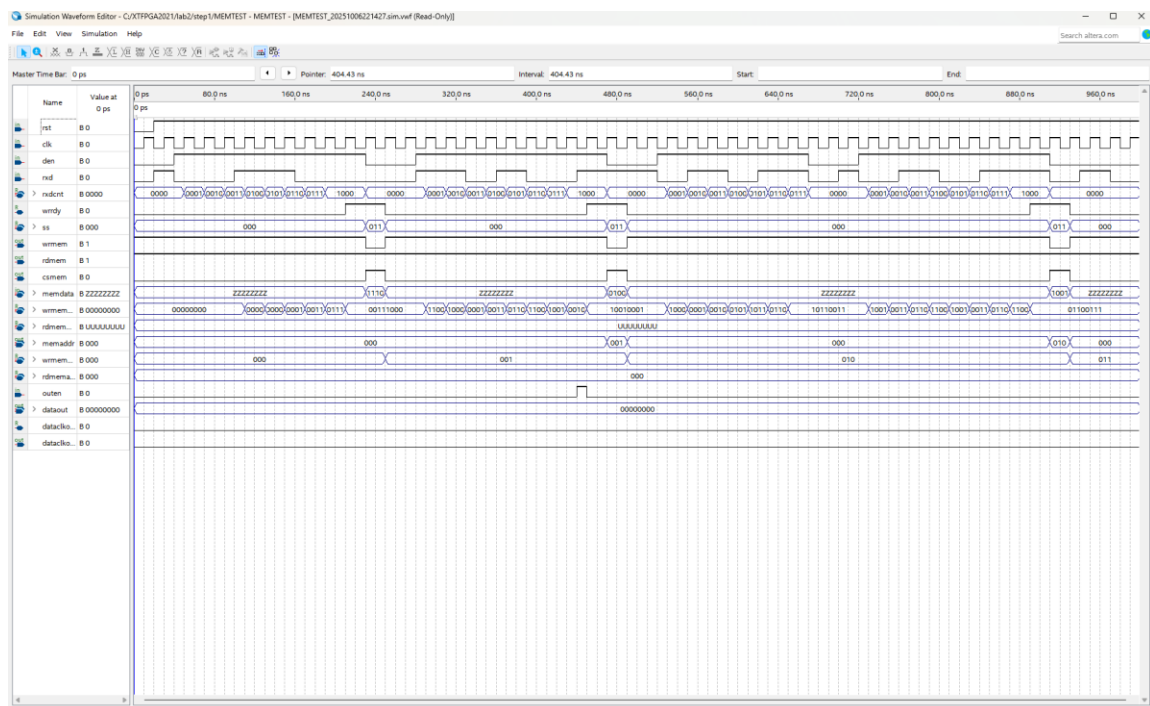
```

158
159 -- 进程7: 数据输出寄存器
160 process(rst,clk)
161 begin
162     if rst = '0' then
163         dataout <= (others => '0');
164     elsif clk'event and clk = '1' then
165         if ss = s2 then -- 读状态时将内存数据输出
166             dataout <= rdmemdata;
167         end if;
168     end if;
169 end process;
170
171 -- 进程8: 与地址计数器
172 process(rst,clk)
173 begin
174     if rst = '0' then
175         wrmemaddr <= (others => '0');
176     elsif clk'event and clk = '1' then
177         if ss = s1 then -- 每次与操作后地址递增
178             wrmemaddr <= wrmemaddr + 1;
179         end if;
180     end if;
181 end process;
182
183 -- 进程9: 读地址计数器
184 process(rst,clk)
185 begin
186     if rst = '0' then
187         rdmemaddr <= (others => '0');
188     elsif clk'event and clk = '1' then
189         if ss = s2 then -- 每次读操作后地址递增
190             rdmemaddr <= rdmemaddr + 1;
191         end if;
192     end if;
193 end process;
194
195 -- 组合逻辑: 地址多路选择器
196 memaddr <= wrmemaddr when wrmem_s = '0' else rdmemaddr; -- 根据读与选择地址源
197
198 -- 组合逻辑: 内存数据总线控制
199 memdata <= wrmemdata when wrmem_s = '0' else "zzzzzzzz"; -- 与时输出数据, 读时高阻态
200 rdmemdata <= memdata; -- 读操作时从内存总线读取数据
201 wrmem <= wrmem_s; -- 与内存控制信号输出
202
203 end behav;
204
205
206
207

```

3. 根据 memtest.vhd 代码重新完善系统框图。

4. 仿真 memtest.vhd 代码。



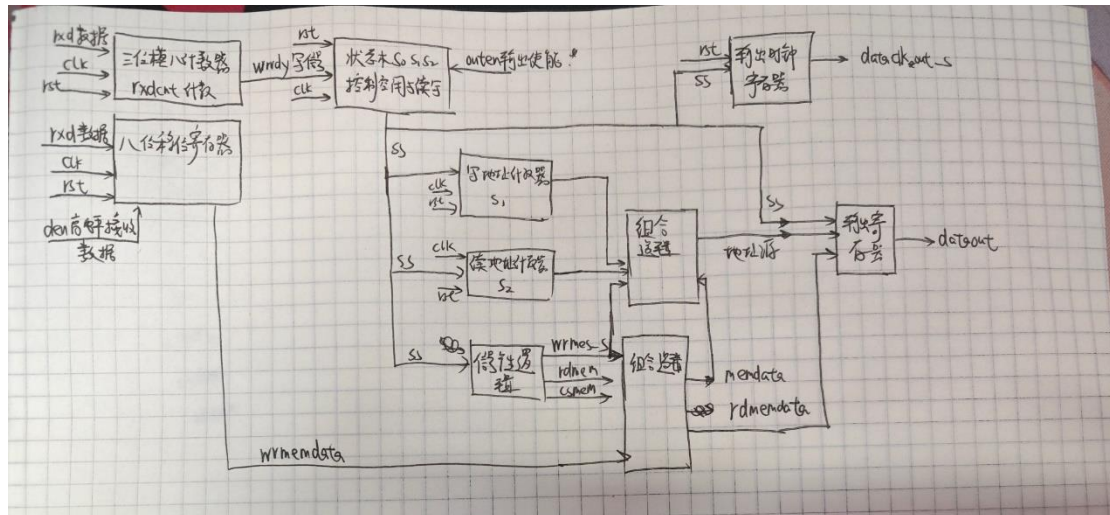
三、port map () 的第二种写法和 MEMTEST 的电路结构细节图

memtest port map () 的第二种写法：

memtest_inst : memtest

```
port map (  
    rst          => rst,  
    clk          => clk,  
    den          => den,  
    rxd          => rxd,  
    outen        => outen,  
    rdmem        => rdmem,  
    wrmem        => wrmem,  
    csmem        => csmem,  
    memdata      => memdata,  
    memaddr      => memaddr,  
    dataout      => dataout,  
    dataclkout   => dataclkout  
);
```

MEMTEST 的电路结构细节图：



四、思考题：状态机设计

在上一个实验中，交通灯代码是一个独立状态机的示例。该例子独立控制单元状态机接受外来输入和信息处理单元状态，输出控制命令指挥信息处理单元工作。相比密码锁、洗衣机、电梯、售货机，等小例子都是同类。

请设计一个两层电梯的控制器代码或者一个支持两种货物，两种货币的售货机。代码自己写、从网上找、用 AI 生成，相互辅助对照，认真分析学习，实现仿真。

电梯设计：

电梯一共两层，一共有 4 个按键：1 楼外只有向上按键，2 楼外只有向下按键，电梯内还有 2 个按键分别为 1 楼按键和 2 楼按键。将所有楼层外和电梯内的按键视为电平信号作为给电梯的运行请求信号。电梯有三个运行状态：待机、上行、下行。

待机：电梯停在 1 楼或 2 楼且无请求信号时均为待机状态。

上行状态：电梯停在 1 楼，有 2 楼外向下按钮或带电梯内 2 楼按钮被按下，进入上行状态。

下行状态：电梯停在 2 楼，有电梯内 1 楼按钮或 1 楼外向上按钮被按下，进入下行状态。

每一层楼之间的运行时间间隔为 5 个时钟周期。

代码实现：

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity elevator is
6  port (
7      clk          : in  STD_LOGIC;
8      reset        : in  STD_LOGIC;
9      up_request   : in  STD_LOGIC;  -- 1楼外向上按钮
10     down_request  : in  STD_LOGIC;  -- 2楼外向下按钮
11     inside_1      : in  STD_LOGIC;  -- 电梯内1楼按钮
12     inside_2      : in  STD_LOGIC;  -- 电梯内2楼按钮
13     current_floor : out STD_LOGIC;  -- 当前楼层: '0'=1楼, '1'=2楼
14     state_out     : out STD_LOGIC_VECTOR(1 downto 0) -- 状态输出: 00=待机, 01=上行, 10=下行
15 );
16 end elevator;
17
18 architecture Behavioral of elevator is
19     -- 状态定义
20     type state_type is (IDLE, UP, DOWN);
21     signal current_state, next_state : state_type;
22     signal floor_reg, next_floor : STD_LOGIC;
23     signal timer : unsigned(2 downto 0); -- 0-4计数
24     signal timer_enable : STD_LOGIC;
25     signal request_up : STD_LOGIC;  -- 上行请求
26     signal request_down : STD_LOGIC; -- 下行请求
27 begin
28     -- 组合请求信号
29     request_up <= (down_request or inside_2); -- 2楼外向下或电梯内2楼按钮
30     request_down <= (up_request or inside_1); -- 1楼外向上或电梯内1楼按钮
31
32
33
34
```

```

37
38 -- 状态寄存器
39 process(clk, reset)
40 begin
41     if reset = '1' then
42         current_state <= IDLE;
43         floor_reg <= '0'; -- 复位时停在1楼
44         timer <= (others => '0');
45     elsif rising_edge(clk) then
46         current_state <= next_state;
47         floor_reg <= next_floor;
48         -- 计时器逻辑
49         if timer_enable = '1' then
50             if timer = 4 then
51                 timer <= (others => '0');
52             else
53                 timer <= timer + 1;
54             end if;
55         else
56             timer <= (others => '0');
57         end if;
58     end if;
59 end process;
60
61 -- 下一状态和输出逻辑
62 process(current_state, floor_reg, request_up, request_down, timer)
63 begin
64     -- 默认值
65     next_state <= current_state;
66     next_floor <= floor_reg;
67     timer_enable <= '0';
68     case current_state is
69         when IDLE =>
70             if floor_reg = '0' then -- 当前在1楼
71                 if request_up = '1' then
72                     next_state <= UP;
73                     timer_enable <= '1';
74                 end if;
75             else -- 当前在2楼
76                 if request_down = '1' then
77                     next_state <= DOWN;
78                     timer_enable <= '1';
79                 end if;
80             end if;
81         when UP =>
82             timer_enable <= '1';
83             if timer = 4 then -- 完成5个时钟周期的上行
84                 next_state <= IDLE;
85                 next_floor <= '1'; -- 到达2楼
86             end if;
87         when DOWN =>
88             timer_enable <= '1';
89             if timer = 4 then -- 完成5个时钟周期的下行
90                 next_state <= IDLE;
91                 next_floor <= '0'; -- 到达1楼
92             end if;
93         end case;
94     end process;
95
96 -- 输出分配
97 current_floor <= floor_reg;
98
99 with current_state select
100     state_out <= "00" when IDLE,
101                 "01" when UP,
102                 "10" when DOWN,
103                 "00" when others;
104 end Behavioral;
105
106
107
108
109
110
111

```

我们将电梯分为三个运行状态，用来描述状态机的值：

IDLE：待机状态 00

UP：上行状态 01

DOWN：下行状态 10

通过以下四个输入信号来模拟电梯的四个按键接受情况：

up_request: 1 楼外向上按钮

down_request: 2 楼外向下按钮

inside_1: 电梯内 1 楼按钮

inside_2: 电梯内 2 楼按钮

同时给出两个状态输出：

current_floor: 当前楼层 (0=1 楼, 1=2 楼)

state_out: 当前状态编码

关于计时机制，使用 3 位计时器计数 0-4，在 UP 或 DOWN 状态时使能计时器，当计数到 4 时完成楼层间移动

关于三个状态的转换逻辑：

1 楼待机时，收到上行请求进入 UP 状态，2 楼待机时，收到下行请求进入 DOWN 状态，运行 5 个时钟周期后返回待机状态并更新楼层。

我们将四个按键信号用组合逻辑得到上升信号 request_up 和下降信号

request_down:

$\text{request_up} \leq (\text{down_request} \text{ or } \text{inside_2});$

$\text{request_down} \leq (\text{up_request} \text{ or } \text{inside_1});$

则可以归纳出状态机的状态变换为：

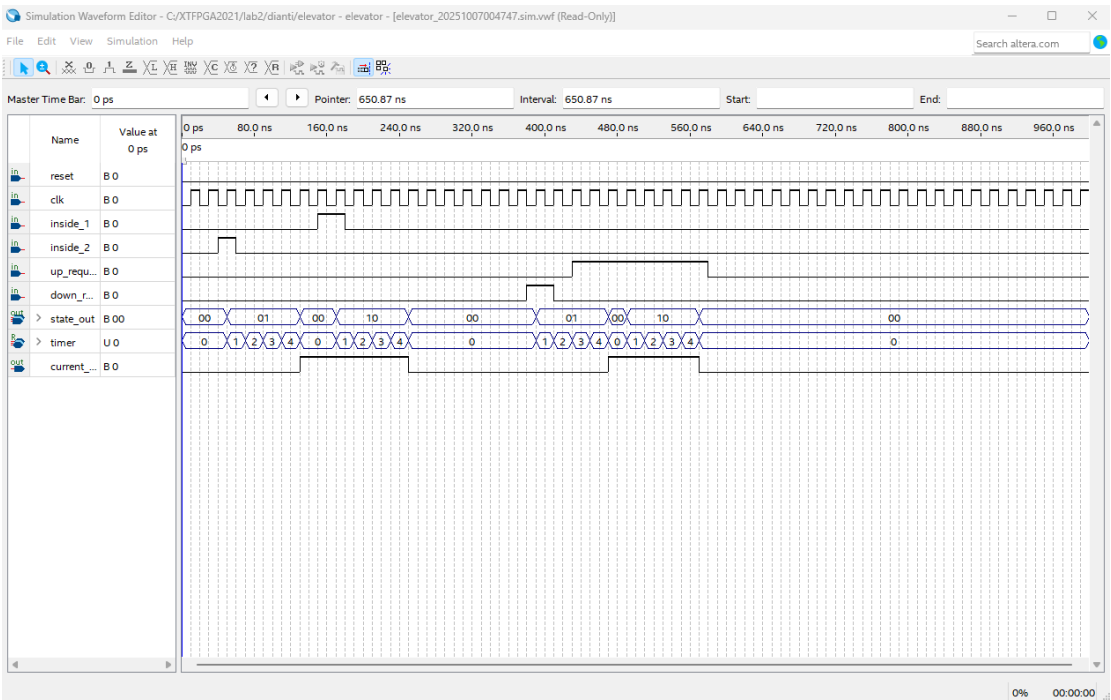
当电梯处于待机状态 00 时：

若当前楼层为一，且有上升信号，则变为 01 上升状态；若当前楼层为二，且有下降信号，则变为 10 下降状态，否则保持待机状态。

当电梯处于上升状态 01 或下降状态 10 时，计数器计数，并完成楼层数从 1

到 2 和从 2 到 1 的变化，即更新 `current_floor`。

仿真结果：



library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

use IEEE.NUMERIC_STD.ALL;

entity elevator is

Port (

clk : in STD_LOGIC;

reset : in STD_LOGIC;

up_request : in STD_LOGIC; -- 1 楼外向上按钮

down_request : in STD_LOGIC; -- 2 楼外向下按钮

inside_1 : in STD_LOGIC; -- 电梯内 1 楼按钮

inside_2 : in STD_LOGIC; -- 电梯内 2 楼按钮

```
current_floor: out STD_LOGIC; -- 当前楼层 :&apos;0&apos;=1  
楼, &apos;1&apos;=2 楼
```

```
state_out : out STD_LOGIC_VECTOR(1 downto 0) -- 状态输出: 00=待  
机, 01=上行, 10=下行
```

```
);
```

```
end elevator;
```

```
architecture Behavioral of elevator is
```

```
-- 状态定义
```

```
type state_type is (IDLE, UP, DOWN);
```

```
signal current_state, next_state : state_type;
```

```
-- 当前楼层信号
```

```
signal floor_reg, next_floor : STD_LOGIC;
```

```
-- 计时器信号
```

```
signal timer : unsigned(2 downto 0); -- 0-4 计数
```

```
signal timer_enable : STD_LOGIC;
```

```
-- 请求信号组合
```

```
signal request_up : STD_LOGIC; -- 上行请求
```

```
signal request_down : STD_LOGIC; -- 下行请求
```

```

begin

    -- 组合请求信号

    request_up <= (down_request or inside_2);    -- 2 楼外向下或电梯内 2 楼按钮
    request_down <= (up_request or inside_1);    -- 1 楼外向上或电梯内 1 楼按钮

    -- 状态寄存器

    process(clk, reset)
    begin
        if reset = '1' then
            current_state <= IDLE;
            floor_reg <= '0';    -- 复位时停在 1 楼
            timer <= (others => '0');
        elsif rising_edge(clk) then
            current_state <= next_state;
            floor_reg <= next_floor;

            -- 计时器逻辑

            if timer_enable = '1' then
                if timer = 4 then

```

```

        timer <= (others => '0');

    else

        timer <= timer + 1;

    end if;

else

    timer <= (others => '0');

    end if;

end if;

end process;

-- 下一状态和输出逻辑

process(current_state, floor_reg, request_up, request_down, timer)

begin

    -- 默认值

    next_state <= current_state;

    next_floor <= floor_reg;

    timer_enable <= '0';

    case current_state is

        when IDLE =>

            if floor_reg = '0' then -- 当前在 1 楼

                if request_up = '1' then

```

```

        next_state <= UP;

        timer_enable <= '1';

    end if;

else -- 当前在 2 楼

    if request_down = '1' then

        next_state <= DOWN;

        timer_enable <= '1';

    end if;

end if;

when UP =>

    timer_enable <= '1';

    if timer = 4 then -- 完成 5 个时钟周期的上行

        next_state <= IDLE;

        next_floor <= '1'; -- 到达 2 楼

    end if;

when DOWN =>

    timer_enable <= '1';

    if timer = 4 then -- 完成 5 个时钟周期的下行

        next_state <= IDLE;

        next_floor <= '0'; -- 到达 1 楼

```

```
end if;
```

```
end case;
```

```
end process;
```

```
-- 输出分配
```

```
current_floor <= floor_reg;
```

```
with current_state select
```

```
state_out <= "00" when IDLE,
```

```
        "01" when UP,
```

```
        "10" when DOWN,
```

```
        "00" when others;
```

```
end Behavioral;
```